

Interview Questions For Fundamentals of Programming

Theory Interview Questions: Fundamentals of Programming

1. **What is a program?**
 - Explain what a program is and why it is important in software development.
2. **What is the difference between a compiler and an interpreter?**
 - Discuss the key differences and examples of compiled (e.g., C) and interpreted (e.g., Python) languages.
3. **What is an algorithm? Why is it important in programming?**
 - Define an algorithm and explain its role in problem-solving and software development.
4. **Explain the concept of variables in programming.**
 - What is a variable? How are variables used to store data in a program?
5. **What are data types in programming?**
 - List common data types (int, float, char, boolean) and explain how they are used.
6. **What is a control flow statement?**
 - Explain control flow statements like if-else, switch, for loops, and while loops.
7. **What is a function?**
 - Define functions and explain their importance in writing reusable code.
8. **Explain the concept of recursion.**
 - What is recursion, and how is it different from iteration? Provide a simple use case.
9. **What are arrays in programming?**
 - Define arrays and explain how they are used to store multiple values.
10. **What is object-oriented programming (OOP)?**
 - Explain the main concepts of OOP: encapsulation, inheritance, polymorphism, and abstraction.
11. **What is the difference between procedural programming and object-oriented programming?**
 - Discuss key differences and provide examples of each approach.
12. **What is exception handling?**
 - Explain how exception handling works and why it is important for building robust applications.
13. **What is the difference between a stack and a queue?**
 - Define stacks and queues, and explain how they differ in terms of data storage and access.
14. **What are the key principles of software development?**
 - Discuss principles like DRY (Don't Repeat Yourself), KISS (Keep It Simple, Stupid), and modularity.
15. **What are the common types of errors in programming?**
 - Discuss syntax errors, runtime errors, and logical errors, and explain how they can be addressed.

Practical Interview Questions: Fundamentals of Programming

1. **Write a program to print “Hello, World!” on the screen.**
 - A simple test to check if the candidate understands basic syntax and output.
 2. **Write a program to swap two numbers without using a third variable.**
 - Test for logic building and use of arithmetic operations.
 3. **Write a program to check whether a number is even or odd.**
 - Simple conditional statements to test understanding of if-else logic.
 4. **Write a program to find the largest of three numbers.**
 - Test for conditional and comparison operators.
 5. **Write a program to calculate the factorial of a number using recursion.**
 - Test for understanding of recursion.
 6. **Write a program to reverse a string without using built-in functions.**
 - Test string manipulation and logic skills.
 7. **Write a program to print the Fibonacci series up to a certain number.**
 - Test for loop and recursion concepts.
 8. **Write a program to implement a simple calculator that performs addition, subtraction, multiplication, and division.**
 - Test for input handling, conditionals, and arithmetic operations.
 9. **Write a program to sort an array of integers using bubble sort.**
 - Test understanding of sorting algorithms.
 10. **Write a program to count the number of vowels in a string.**
 - Test for string handling and iteration through arrays.
 11. **Write a program to find whether a given number is prime or not.**
 - Test for loops and conditional logic.
 12. **Write a program to find the sum of all elements in an array.**
 - Test for array handling and looping.
 13. **Write a program to implement binary search on a sorted array.**
 - Test for understanding of searching algorithms.
 14. **Write a program to find the greatest common divisor (GCD) of two numbers using recursion.**
 - Test for recursion and logic building.
 15. **Write a program to reverse an integer.**
 - Test for number manipulation.
-

Advanced Practical Questions (for Stronger Freshers)

1. **Write a program to implement a stack using arrays or linked lists.**
 - Test for data structures understanding and how to manipulate them.
 2. **Write a program to implement a queue using arrays or linked lists.**
 - Another data structure problem to test understanding of enqueue and dequeue operations.
 3. **Write a program to implement a binary search tree (BST).**
 - Test for more advanced understanding of tree structures.
 4. **Write a program to solve the Tower of Hanoi problem using recursion.**
 - Test for recursion and problem-solving ability.
 5. **Write a program to implement a basic text-based Tic-Tac-Toe game.**
 - Test problem-solving, use of arrays, loops, and conditionals.
-

Scenario-based Programming Questions

1. **Write a program to simulate an ATM withdrawal.**
 - The candidate should be able to handle user input, validate balances, and update amounts.
2. **Write a program that reads a file and counts the number of words in it.**
 - Test for file handling and string manipulation.
3. **Write a program that takes user input for a matrix and prints its transpose.**
 - Test understanding of 2D arrays and loops.
4. **Write a program that simulates a simple banking system with account creation, deposit, and withdrawal functionalities.**
 - Test object-oriented programming skills and logic building.
5. **Write a program to manage student records using classes (create, read, update, and delete operations).**
 - Test OOP, file handling, and data management.

Theory Interview Questions: Database Management Systems (DBMS)

1. **What is a Database?**
 - Define what a database is and its importance in storing and managing data.
2. **What is DBMS?**
 - Explain what a Database Management System (DBMS) is and its role in database management.
3. **What are the types of DBMS?**
 - List and explain the four types of DBMS (Hierarchical, Network, Relational, Object-oriented).
4. **What is the difference between a DBMS and an RDBMS?**
 - Discuss the main differences between a standard DBMS and a Relational DBMS (RDBMS).
5. **What is a Primary Key?**
 - Explain the concept of a primary key and its importance in uniquely identifying records in a table.
6. **What is a Foreign Key?**
 - Define a foreign key and its role in establishing relationships between tables.
7. **What is Normalization?**
 - Explain the process of normalization and why it is necessary in database design. Discuss various normal forms (1NF, 2NF, 3NF, BCNF).
8. **What is Denormalization?**
 - Explain denormalization and when it is used in database design.
9. **What are the ACID properties in DBMS?**
 - Discuss the four ACID properties (Atomicity, Consistency, Isolation, Durability) that ensure reliable transactions.
10. **What is a SQL query?**
 - Explain what SQL is and how it is used to interact with databases. Mention basic types of SQL queries (DDL, DML, DCL, TCL).
11. **What are Constraints in DBMS?**
 - Define constraints (Primary Key, Foreign Key, Unique, Not Null, Check, and Default) and their importance.
12. **What is a Join in SQL?**

- Define joins and explain the different types of joins (INNER, LEFT, RIGHT, FULL).
 - 13. **What is a View in SQL?**
 - Explain what a view is, how it is created, and its advantages in database management.
 - 14. **What are Indexes in SQL?**
 - Define indexing and explain how it improves database query performance.
 - 15. **What is the difference between DELETE, TRUNCATE, and DROP commands in SQL?**
 - Explain the key differences between these three commands and their impact on data and database structure.
 - 16. **What is a Stored Procedure?**
 - Define stored procedures and explain how they are used to improve performance and security in database operations.
 - 17. **What is a Trigger in DBMS?**
 - Explain what triggers are, how they work, and their use cases in a database system.
 - 18. **What is a Transaction in DBMS?**
 - Define a transaction and explain the importance of maintaining transaction integrity.
 - 19. **What is SQL Injection?**
 - Explain SQL injection and how to prevent it.
 - 20. **What is a Cursor in DBMS?**
 - Define cursors and explain their role in fetching data row by row from a result set.
-

Practical Interview Questions: Database Management Systems (DBMS)

1. **Write a SQL query to create a table for storing employee details with fields such as Employee ID, Name, Department, and Salary.**
 - Test basic SQL syntax for creating tables and defining data types.
2. **Write a SQL query to insert multiple records into the employee table.**
 - Check the ability to use `INSERT INTO` to populate tables with data.
3. **Write a SQL query to update the salary of all employees in the 'IT' department by 10%.**
 - Test for understanding of the `UPDATE` command with conditions.
4. **Write a SQL query to delete all employees whose salary is less than 5000.**
 - Test for the ability to delete data using the `DELETE` command with conditions.
5. **Write a SQL query to retrieve all employees who work in the 'HR' department.**
 - Test the candidate's ability to use the `SELECT` statement with `WHERE` conditions.
6. **Write a SQL query to display the total number of employees in each department.**
 - Test for the ability to use aggregate functions (`COUNT`, `GROUP BY`).
7. **Write a SQL query to find the second highest salary in the employee table.**
 - Test problem-solving skills and the use of `LIMIT`, `ORDER BY`, or subqueries.
8. **Write a SQL query to join the Employee table with the Department table and fetch the employee's name along with their department name.**
 - Test for understanding of `JOIN` operations (INNER, LEFT, RIGHT).

9. **Write a SQL query to create a view that displays all employees along with their department names.**
 - Check for knowledge of how to create and manage views.
10. **Write a SQL query to fetch employees with duplicate names in the employee table.**
 - Test for problem-solving and the use of `GROUP BY` with `HAVING` clause.
11. **Write a stored procedure to insert a new employee into the employee table.**
 - Check the candidate's ability to write and use stored procedures.
12. **Write a trigger that automatically updates the employee's salary after a certain period of time (e.g., every year).**
 - Test understanding of triggers and their practical use in real-world applications.
13. **Write a SQL query to retrieve employee details along with the department name for employees whose salary is greater than 5000.**
 - Test for the ability to use joins with conditions and filtering.
14. **Write a SQL query to fetch the employees in ascending order of their department and descending order of their salary.**
 - Test for ordering and sorting queries using `ORDER BY`.
15. **Write a SQL query to perform an inner join on two tables (e.g., Employee and Department) and select all employees with a salary above the average salary.**
 - Test for combining `JOIN` with aggregate functions.
16. **Write a SQL query to create an index on the employee's salary column and explain how indexing improves query performance.**
 - Test for knowledge of indexing and performance optimization.
17. **Write a SQL query to create a foreign key between the Employee table and the Department table.**
 - Check for understanding of foreign keys and how relationships are defined between tables.
18. **Write a SQL query to demonstrate the use of a cursor to fetch one row at a time from a result set.**
 - Test for advanced querying using cursors.
19. **Create a transaction where a salary deduction is applied, and if any error occurs, the transaction should be rolled back.**
 - Test the ability to work with transactions and error handling (`BEGIN TRANSACTION`, `COMMIT`, `ROLLBACK`).
20. **Write a SQL query to generate a report of employee salaries in each department using a subquery.**
 - Test for the ability to use subqueries in complex queries.